

#ENTERPRISE AI & DATA STRATEGY

Oye k8s?

**Cómo está el cluster:
operaciones conversacionales
con IA**

Christian Vargas
Diego Del Castillo
+ Cloud IT



Perfil del speaker



Christian Vargas

Solutions Architect @ +Cloud IT



Diego del Castillo

Solutions Architect @ +Cloud IT

DEVOPSDAYS.PE



El problema:

**Son las 03:00 horas
y algo no responde.**



```
$ kubectl get pods -A | grep -i crash  
$ kubectl describe pod checkout-7f9d... -n demo  
$ kubectl logs checkout-7f9d... --previous  
$ kubectl get events --sort-by=.lastTimestamp  
$ kubectl rollout history deploy/checkout
```

El coste real no son
los comandos.

**Es el cambio de contexto
mental entre cinco sitios
distintos.**

La alternativa:



La misma pregunta, en una frase

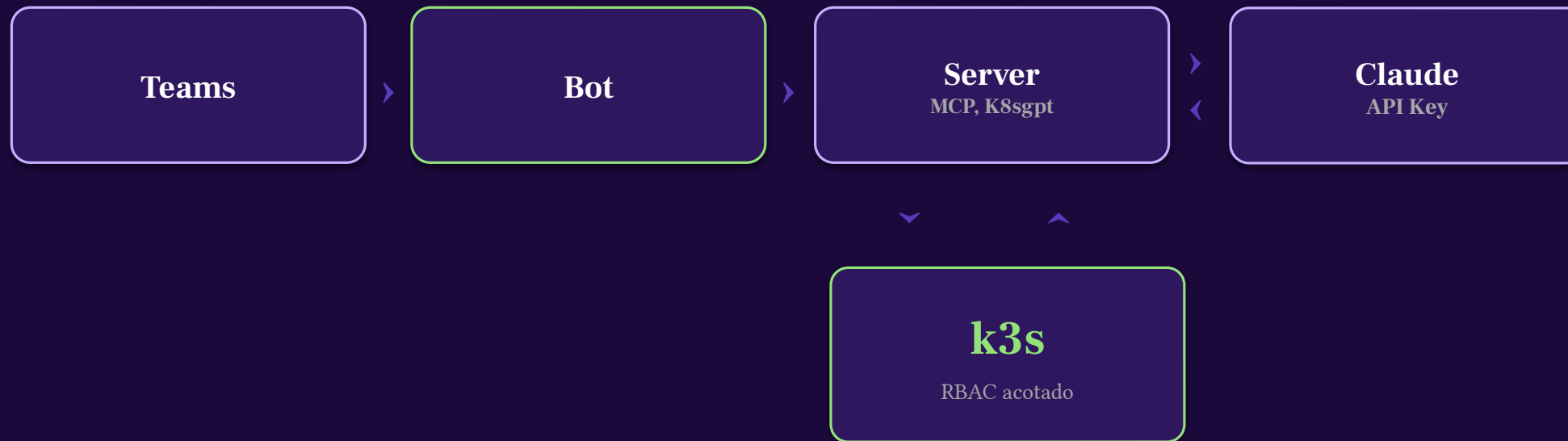
Oye K8s, ¿Por qué está fallando checkout-service?

El bot encadena las herramientas de lectura que hagan falta: pods, logs, eventos, historial de rollout y responde con la causa raíz y desde cuándo.

En Teams, donde el equipo ya conversa.

Cómo funciona?

Cinco piezas, una sola pública



Los tres servicios MCP viven en localhost. Nunca se exponen a internet: lo único público es el endpoint del bot.



El concepto que hay que entender:



Una “tool” es una función que la IA puede PEDIR ejecutar.

No algo que ejecuta por su cuenta.

Quien decide si esa petición se convierte en una llamada real al cluster es tu código. Ahí es donde se pone o no se pone la seguridad.



El problema no es la IA.
Es el permiso.

Tres capas independientes:

Ninguna depende de que el modelo se porte bien

01 Catálogo

El bot decide qué herramientas entran en la petición. las de escritura no van. No es una instrucción del prompt: es el JSON que se envía.

02 Servidor MCP

La instancia de diagnóstico corre con --read-only. Filtra al registrar, así que las destructivas no aparecen ni en tools/list.

03 RBAC

La identidad del cluster no tiene el permiso. Aunque fallaran las dos capas anteriores, el API server responde 403.

Empezar en solo lectura y abrir lo mínimo. Es al revés de como se monta normalmente.



**El modelo nunca recibe una
herramienta de escritura.
Ni antes de confirmar, ni después.**

Cuando pulsas Confirmar, ejecuta el bot y ejecuta exactamente el comando que aparecía en la tarjeta. El modelo solo redacta el resultado.



El bootstrap lo verifica solo, contra el API server

- ✓ reader puede listar pods en todo el cluster yes
- ✓ reader NO puede leer secrets no
- ✓ writer puede actualizar deployments en demo yes
- ✓ writer NO puede borrar namespaces no
- ✓ writer NO puede tocar kube-system no
- ✓ writer NO puede hacer exec en pods no
- ✓ writer NO puede borrar deployments en demo no

Un Role acotado al namespace, sin el verbo delete sobre ningún recurso.

Demo

checkout-service en CrashLoopBackOff.

Diagnóstico · el “no” · el arreglo con permiso.



Cuándo NO usar esto

- **Alucinación en contexto operativo.** El diagnóstico puede sonar seguro y estar equivocado.
- **Latencia.** No es para incident response de segundos.
- **Coste de tokens** si lo dejas escuchando todo el tiempo.
- **Nunca producción sin confirmación humana** para lo destructivo.
- **Un servidor MCP por cluster,** fijado a un kubeconfig. Sin ambigüedad.

Esto es lo que separa una charla de vendedor de una charla de ingeniero.



**La conversación no
reemplaza el runbook.
Lo hace accesible.**





Gracias